Санкт-Петербургский государственный университет

Искусственный интеллект и наука о данных

Шевцов Анатолий Владимирович

АВТОМАТИЧЕСКОЕ ВЫЯВЛЕНИЕ СЕМАНТИЧЕСКИХ ОШИБОК ФИКСИРОВАННЫХ ПРАВИЛ КАРТОЧНОЙ ИГРЫ НА ОСНОВЕ РЕСУРСНЫХ ОГРАНИЧЕНИЙ И СИМУЛЯЦИИ

Отчёт о прохождении
Учебной практики

Научный руководитель:

Преподаватель, С.И Салищев

Санкт-Петербург

2026

Содержание

ВВЕДЕНИЕ 3
1 Постановка задачи 5
2 АНАЛИЗ ПРОБЛЕМЫ И СУЩЕСТВУЮЩИХ РЕШЕНИЙ 6
2.1 Определения и различие уровней корректности 6
2.2 Научный контекст и ограничения существующих подходов 7
2.3 Конкретная проблема и входные данные валидатора 9
2.4 Исследовательский вклад и критерии результата 10
3 ФОРМАЛЬНАЯ МОДЕЛЬ ИГРЫ 12
3.1 Состояния, действия и правила карточной дуэли 12
3.2 Предметный контракт и проверяемое свойство 13
4 МЕТОД СТАТИЧЕСКОЙ И СИМУЛЯЦИОННОЙ ДИАГНОСТИКИ 14
4.1 Архитектура и последовательность проверок 14
4.2 Доказуемая верхняя граница урона 14
4.3 Алгоритм динамического программирования 15
4.4 Три статуса и смысл результатов симуляции 16
5 ПРОВЕДЕНИЕ ЭКСПЕРИМЕНТА 17
5.1 Реализация и контроль корректности 17
5.2 Контрольный набор и методика сравнения 17
6 РЕЗУЛЬТАТЫ И ИХ ИНТЕРПРЕТАЦИЯ 19
6.1 Общая сравнительная оценка 19
6.2 Доказательство на примере mana_bottleneck 20
6.3 Контрпример strategy_trap и роль агентов 21
6.4 Что действительно доказал эксперимент 22
7 ОГРАНИЧЕНИЯ И ПРАКТИЧЕСКОЕ ЗНАЧЕНИЕ 23
ЗАКЛЮЧЕНИЕ 25
СПИСОК ИСПОЛЬЗОВАННЫХ ИСТОЧНИКОВ 26
ПРИЛОЖЕНИЕ А. ПРИМЕР ВХОДНОГО ДОКУМЕНТА 28
ПРИЛОЖЕНИЕ Б. ВОСПРОИЗВОДИМОСТЬ 29

ВВЕДЕНИЕ

Игровые правила задают не только набор допустимых полей конфигурации и общую структуру, но и поведение игры — какие действия может совершить игрок, как изменяются здоровье и ресурсы, достижима ли цель. Даже структурно корректный набор правил способен создать игру, в которой победить будет невозможно. Например, наносящая урон карта в рамках карточной игры может быть записана в допустимом JSON, но стоить больше доступной маны; другой вариант — несколько допустимых атак формально существуют, однако в пределах ограниченного числа ходов игрок не успевает оплатить их в количестве, необходимом для победы. При этом простая проверка типов и диапазонов такие нарушения не обнаруживает.

Актуальность и практическая важность. Полная ручная проверка правил требует рассматривать взаимодействия карт, ограничений ресурсов, начальных состояний и действий игроков. По мере расширения набора механик число комбинаций возрастает, и заранее прописывать отдельную эвристику под каждый случай становится достаточно трудоёмко. Автоматический плейтестинг позволяет получать наблюдения о поведении игры, но создаёт вторую проблему: если тестовый агент ни разу не победил, это может объясняться слабостью стратегии, а не дефектом самих правил. Ошибочный вывод «нет побед в тестах — правила некорректны» приводит к необоснованной отбраковке допустимых вариантов.

Состояние исследований. Существуют методы проверки и создания игровых механик с применением ограничений, планирования и логического вывода [2–5], автоматического плейтестинга и агентной оценки [1, 6, 10, 11], а также методы валидации игрового контента на основании статистики прохождений [12]. Поэтому сама идея симуляции игровых правил не является новой. Однако в прикладной процедуре важно различать логическое доказательство недостижимости и эмпирическое отсутствие успеха конкретного агента; в противном случае два разных вывода могут ошибочно превращаться в один бинарный вердикт.

Исследовательская проблема настоящей работы формулируется узко: как для заранее заданной карточной дуэли автоматически обнаруживать определённые ошибки игровых правил и при этом не объявлять невозможной победу только на основании неуспешных партий тестового агента? Для ответа используется комбинация двух средств: формальной оптимистической оценки достижимого урона и поиска конкретных выигрышных траекторий при симуляции.

Идея и новизна в границах учебной практики. Для фиксированной игровой модели разработана объяснимая процедура с тремя исходами: (1) доказанное нарушение контракта или необходимого ресурсного условия; (2) конкретная наблюдаемая победа; (3) отсутствие достаточного основания для решения. В качестве технического расширения ранних статических проверок реализована верхняя граница суммарного урона, учитывающая число ходов, общий запас получаемой маны и экземпляры карт. Продемонстрировано, что она выявляет ресурсную ошибку, пропущенную более грубой эвристикой, а неопределённый статус предотвращает ложное отрицательное заключение на специальном контрпримере.

Уточнение роли LLM. В предыдущей версии практики модель Qwen3-8B применялась для генерации JSON-конфигураций. Однако проверяемый результат не зависит от того, кто создал исходный набор правил — человек, генератор на основе ограничений либо языковая модель. Поэтому по замечанию преподавателя генерация и верификация LLM исключены из актуального эксперимента: в систему поступают только заранее составленные конфигурации. Это позволяет исследовать именно валидацию и не приписывать LLM свойства, которые в работе не проверялись.

1 Постановка задачи

Цель работы — разработать и экспериментально проверить метод диагностики семантических нарушений в фиксированных правилах карточной дуэли, отделяющий доказанную невозможность штатной победы от неуспеха конкретной игровой стратегии.

Для её достижения решаются следующие задачи:

1. Зафиксировать правила минимальной карточной дуэли и структурную схему входной конфигурации.
2. Определить предметный контракт и требование достижимости штатной наступательной победы.
3. Разработать и реализовать достаточное условие ресурсной недостижимости на основе оптимистической верхней оценки урона.
4. Сохранить симуляцию как средство поиска свидетельств побед/поражений, регистрируя причины завершения отдельно.
5. Провести эксперимент на контролируемых корректных и дефектных конфигурациях, сравнить с базовыми проверками и указать границы результатов.

Объект исследования — формализованные конфигурации правил двухпользовательской карточной игры. Предмет исследования — контрактные и ресурсные ограничения, определяющие корректность и достижимость предусмотренной цели.

Проверяемая гипотеза: добавление доказуемой ресурсной верхней оценки к предметным проверкам обнаруживает ошибки, пропущенные ранними эвристиками; при этом использование отдельного статуса неопределённости исключает логически необоснованное отклонение правил по нулю побед тестового агента.

В рамках работы не решаются задачи обучения агента, автоматического восстановления повреждённых правил, оценки интересности и баланса, исследования всех возможных стратегий, генерации правил с помощью LLM и верификации языковой модели. Эти задачи могут входить в будущую систему, но не служат основанием нынешних выводов.

2 АНАЛИЗ ПРОБЛЕМЫ И СУЩЕСТВУЮЩИХ РЕШЕНИЙ

2.1 Определения и различие уровней корректности

Для исключения неоднозначностей различаются следующие понятия.

- Игровая конфигурация — документ, определяющий параметры и свойства карт конкретной реализации карточной дуэли: стартовое здоровье, предел маны, максимальное число раундов, размер колоды и эффекты карт.
- Структурная корректность — соответствие документа формальной схеме данных: допустимые типы полей, их значения и диапазоны. Например, параметр cost должен быть целым неотрицательным числом, параметр effect – принадлежать множеству {damage, heal, draw}.
- Предметная (контрактная) корректность — соответствие содержимого конфигурации заранее установленным смысловым требованиям данной игры. Например, повреждение должно быть направлено на противника, а лечение — на владельца карты.
- Поведенческое свойство — утверждение о возможных последовательностях игровых состояний при выполнении правил: в частности, существует ли допустимая последовательность действий, заканчивающаяся предусмотренной победой.
- Семантическая ошибка в рамках исследования — нарушение предметного контракта или нарушение выбранного, необходимого для игры поведенческого требования. Это не универсальный критерий качества или интересности любой игры.

Формально игра определяется множеством состояний S, множеством допустимых действий A(s), функцией перехода T(s,a), множеством возможных стартовых состояний S_0 и множеством состояний штатной наступательной победы W. Стартовые состояния могут различаться порядком карт в перемешанной колоде. Действия игроков включают использование доступной карты либо пропуск хода. Поведенческое требование в настоящей модели состоит в следующем: должна существовать хотя бы одна допустимая партия из некоторого $s_0 \in S_0$, в которой в пределах R собственных ходов игрок обнулит HP противника своей атакой.

$$P_exist: \exists\ s_0 \in S_0,\ \exists\ a_1, \ldots, a_t :\ T^*(s_0, a_1 \ldots a_t) \in W,\quad t \le R.$$

Важно, что это экзистенциальное требование: достаточно одной допустимой выигрышной траектории для его выполнения. Оно не означает, что каждый игрок побеждает при любой раздаче или стратегии. Напротив, для доказательства недостижимости штатной победы нужно исключить все допустимые траектории рассматриваемого типа. Предложенный в рамках этого исследования ресурсный тест даёт

достаточное условие для такого отрицательного вывода (о недостижимости штатной победы), но не является полным алгоритмом для анализа всех состояний игры.

Таблица 1 – Уровни проверки одних и тех же игровых правил

Уровень	На какой вопрос отвечает	Пример
Структурный	Можно ли разобрать игровую конфигурацию по схеме?	cost=3, effect="damage" допустимы.
Контрактный	Соответствуют ли значения в конфигурации смыслу этой игры?	damage по self нарушает объявленный контракт.
Поведенческий	Возможна ли предусмотренная победная последовательность?	При R=5 и стоимости атак в 3 ед. урона для победы не хватает.
Эмпирический	Удалось ли тестовому агенту найти такую последовательность?	Greedy может не выиграть, хотя победа существует.

Таким образом, структурная корректность необходима для выполнения программы, но не обеспечивает предметную или поведенческую корректность. Кроме того, наблюдение успешной партии является свидетельством существования как минимум одной победной траектории, однако отсутствие такого наблюдения в конечной выборке из сыгранных партий не является доказательством отсутствия победных траекторий вообще.

Отдельно различаются термины «статическая» и «статистическая» проверка. Статическая проверка анализирует правила без выполнения партий (контракт, верхняя оценка максимального урона). Статистическая оценка сводит результаты выполненных партий в доли исходов и среднюю длительность. Эти методы не взаимозаменяемы. Все уровни проверок представлены в Таблице 1.

2.2 Научный контекст и ограничения существующих подходов

Ранние исследования автоматического игрового дизайна уже содержали оценку правил. Browne и Maire [1] использовали самостоятельную игру агентов для оценки качества создаваемых настольных игр; Smith, Nelson и Mateas [2] разработали логический движок Ludocore для представления механик и построения трасс исполнения. Smith и Mateas [3] предложили декларативно описывать пространство возможного контента средствами Answer Set Programming. Эти работы показывают, что формальный язык и автоматическая проверка давно являются частью исследований игр.

Особенно близка работа Zook и Riedl [5] «Automatic Game Design via Mechanic Generation». Авторы представляют механики как операторы планирования, создают допустимые варианты решателем ограничений и проверяют требования к играбельности при помощи планировщика. Это принципиально важно для позиционирования результата:

утверждать, что настоящая практика впервые совместила правила и автоматическую проверку, нельзя. Отличие заключается в предельно узкой ресурсной проверке одной игры и в явном разделении силы отрицательных и положительных свидетельств.

Zook, Fruchter и Riedl [6] рассматривали автоматизацию плейтестинга при настройке параметров с помощью активного обучения. Работа подтверждает трудоёмкость ручной оценки и потребность автоматизировать часть тестов, однако посвящена настройке параметров с участием информации от игроков, а не формальному доказательству недостижимости заданной цели. Общее место ограничений, процедурной генерации и способов оценивания описано в монографии Shaker, Togelius и Nelson [7].

Osborn, Lambrigger и Mateas [8] использовали гибридные автоматы и запросы достижимости для анализа action-игр. Yannakakis и Togelius [9] систематизировали методы игрового ИИ, включая проектирование и оценку. Holmgård и соавторы [10] предложили процедурные персоны и MCTS для тестирования контента с учётом различных игровых стилей; это подчёркивает, что поведение тестового агента является моделью, а не исчерпывающим описанием поведения всех игроков.

В Mechanic Maker 2.0 Gonzalez, Cooper и Guzdial [11] использовали обучение с подкреплением для оценки генерируемых правил и сравнили результаты с агентом A*. Эта работа особенно близка по идее оценивания игровых механик через агентов, однако её задача — получить полезный приближённый показатель качества создаваемых правил. Из такого оценивания не следует возможность интерпретировать любую неуспешную серию игровых партий как доказательство несуществования победной стратегии. Наша работа не воспроизводит и не превосходит количественно [11], а задаёт иной, формально более осторожный тип вывода.

Aylagas и соавторы [12] применили сведения о сложности прохождения для повышения валидности генерируемых уровней для игры Match-3. Их объект исследования — уровень, а не формальный набор механик, но работа подчёркивает практическую пользу автоматизированных игровых испытаний. Она также показывает, почему статистическое качество и достижимость конкретной игровой цели должны быть явно определены в постановке.

Таблица 2 – Сводная таблица публикаций по методам и решаемой задаче

Источник	Метод / объект	Как соотносится с задачей
Browne, Maire (2010) [1]	Самоигра; оценка качества новых игр	Оценка интересности не тождественна доказательству достижимости.

Источник	Метод / объект	Как соотносится с задачей
Smith et al. (2010) [2]	Логическое описание и трассы	Близость по формализации; иная игровая модель.
Smith, Mateas (2011) [3]	ASP и пространство генерации	Создание допустимых вариантов игр, а не наш ресурсный тест.
Zook, Riedl (2014) [5]	Ограничения + планировщик	Главный научный аналог; не заявляется первенство проверки.
Zook et al. (2014) [6]	Автоматизация плейтестинга	Цель — настройка параметров, не доказательство недостижимости победы.
Osborn et al. (2017) [8]	Формальная достижимость	Иная модель действия; направление близко по смыслу.
Holmgård et al. (2019) [10]	Агенты разных стилей	Подчёркивает зависимость результатов наблюдений от агентной политики.
Mechanic Maker 2.0 (2023) [11]	RL и A* при оценке правил	Разделяется эмпирическая оценка и логический вывод.
Aylagas et al. (2024) [12]	Статистика и валидация уровней	Близко по плейтестингу, но другой проверяемый объект.

Вывод из обзора: развитие исследований идёт по двум дополняющим направлениям — строгая проверка формальных свойств правил и поведенческая оценка игры агентами. При этом ограничение агентных результатов состоит в неполном охвате возможных игровых стратегий, а ограничение предметных статических проверок — необходимости формализовать все игровые механики. Предлагаемая процедура сочетает узкое доказуемое условие с симуляцией, но не претендует на проверку всех возможных правил в произвольных играх.

2.3 Конкретная проблема и входные данные валидатора

Рассмотрим проблему на примере одной из проверяемых системой ситуаций, которая выглядит следующим образом. Разработчик заранее задаёт конфигурацию карточной дуэли. Pydantic успешно принимает документ: все поля имеются, эффекты и числовые значения допустимы. Но после запуска симуляции выясняется, что либо игрок не способен оплатить нужное число атакующих карт, либо жадный агент не нашёл способа победить. Само по себе совпадение этих двух случаев с результатом «0 побед из 200» не позволяет утверждать об идентичности причины, их породившей. Нужна проверка, которая отличает ограничение самих правил от ограниченности агента и его стратегии.

На вход системы подаётся один JSON-документ с параметрами игры и описаниями типов карт. Конкретный пример входного документа – сценарий mana_bottleneck – представлен ниже на Рисунке 1. Здесь приведены ключевые поля, полный вариант дан в **Приложении А**.

```
{
  "game_name": "Fixed Card Duel",
  "starting_hp": 10,
  "max_rounds": 5,
  "mana_cap": 3,
  "starting_hand_size": 3,
  "deck_size": 12,
  "win_condition": "hp_zero",
  "cards": [
    {"name":"strike", "cost":3, "effect":"damage", "value":3, "target":"enemy"},
    {"name":"blast",  "cost":3, "effect":"damage", "value":4, "target":"enemy"},
    {"name":"restore","cost":2, "effect":"heal",   "value":2, "target":"self"},
    {"name":"insight","cost":1, "effect":"draw",   "value":1, "target":"self"}
  ]
}
```

Рисунок 1 – Часть примере входного документа сценария mana_bottleneck

Все числовые поля допустимы по схеме. Тем не менее две атаки по три единицы маны за пять ходов оплатить невозможно: всего начисляется не более пяти единиц маны. Ни одна атака не наносит больше четырёх урона; значит, необходимого суммарного урона в 10 HP достичь нельзя. В этом и состоит содержательная проблема, которую структурная проверка не видит.



Рисунок 2 – Вход, последовательность проверок системы и возможные результаты

Проверка, на которую собственно делается акцент в данной работе, находится в двух средних этапах схемы: предметный контракт и ресурсная верхняя граница U. В свою очередь, блок с Pydantic — это базовый этап структурной проверки, а Greedy и Random — дополнительные тестовые агенты, а не часть математического доказательства. Система предназначена для выдачи объяснимого результата с указанием его логической силы.

2.4 Исследовательский вклад и критерии результата

Вклад настоящей учебной практики не состоит в новой универсальной модели ИИ и не требует наличия LLM. Работа демонстрирует три конкретных инженерно-исследовательских результата:

- Ресурсное ограничение, построенное не как отдельное правило для конкретного имени карты, а как оптимизационная задача по всем доступным экземплярам атакующих карт с ограничениями на ход и общую ману.
- Классификацию, в которой доказательство недостижимости наступательной победы, положительное свидетельство и неопределённый результат симуляций не смешиваются в бинарное решение по произвольному отсеивающему порогу.
- Воспроизводимый контрпример, показывающий, что неудача одного агента не означает несуществования выигрышной стратегии.

Критерии успешности:

1) корректная диагностика заранее известных контрактных ошибок;
2) получение формального сертификата $U < H$ для хотя бы одного сценария, пропущенного ранней эвристикой;
3) наличие сценария, где Greedy не побеждает, но альтернативная стратегия предъявляет победу;
4) возможность воспроизвести результаты по фиксированным seed.

Количественное превосходство над опубликованными ранее системами [5] или [11] без совместного эксперимента не заявляется.

3 ФОРМАЛЬНАЯ МОДЕЛЬ ИГРЫ

3.1 Состояния, действия и правила карточной дуэли

Рассматривается симметричная двухпользовательская дуэль с фиксированной колодой[1]. Параметры базовой конфигурации приведены в Таблице 3.

Таблица 3 – Основные параметры фиксированной игровой модели

Параметр	**Обозначение / значение**	**Значение для модели**
Начальное здоровье	H = 10 HP	Для победы нужно снять не менее 10 HP
Начальная мана	0	Исключает стартовый запас ресурса
Прирост маны	1 за собственный ход	Не более R единиц за R ходов
Максимальный запас маны	mana_cap = 3	Карта с cost > 3 не может быть оплачена
Длина партии	R = 12 раундов в базе	Варианты тестов могут изменять R
Размер колоды	12 карт каждому	Копии типов карт конечны
Начальная рука	3 карты	Изымается из колоды при создании руки
Число карт за ход	Не более 1	Ограничение числа атак

Правила совершения хода заданы явно:

- В начале собственного хода игрок получает одну единицу маны, не превышая mana_cap.
- Из своей колоды игрок добирает одну карту, если колода не пуста.
- Игрок может разыграть одну доступную карту при наличии достаточной маны либо пропустить действие.
- После разыгрывания стоимость вычитается из маны, карта удаляется из руки и в колоду не возвращается.
- Эффекты damage, heal и draw изменяют состояние в соответствии с назначением; лечение не поднимает здоровье выше H.
- После каждого действия отдельно фиксируется opponent_hp_zero, self_defeat либо продолжение игры.
- Если за R раундов штатной победы нет, завершается партия с причиной turn_limit — это не эквивалентно доказанному тупиковому исходу.

Структура состояния для каждого игрока включает (hp, mana, hand, deck). Полное состояние также хранит номер раунда, очередь хода и порядок оставшихся карт. Колода формируется циклическим повторением четырёх типов до заданного deck_size;

[1] https://github.com/AnatolyShev/EduPract-2-semester.git

перемешивание проводится генератором random.Random(seed). Разыгранные экземпляры карт не возвращаются в игру.

Таблица 4 – Базовые типы карт

Карта	Стоимость	Эффект	Сила	Цель
strike	1	damage	3	enemy
blast	2	damage	4	enemy
restore	2	heal	2	self
insight	1	draw	1	self

3.2 Предметный контракт и проверяемое свойство

Контракт — это объявленные разработчиком требования к данной игре, а не универсальные законы жанра. Он включает уникальность имён типов карт; условие target = enemy для damage; target = self для heal и draw; положительное значение силы damage и heal; value = 1 для draw. Карта, которая приводит к самоповреждению (уменьшению собственного HP) в другом игровом жанре могла бы быть корректной, однако здесь она будет считаться именно нарушением заранее выбранной спецификации.

Штатная наступательная победа означает событие opponent_hp_zero, т.е. такое событие, при котором здоровье оппонента становится неположительным после собственного атакующего действия. Завершение игровой партии вследствие причины self_defeat не учитывается как достижение этой цели. Проверяемое поведенческое свойство — достижимость такой победы хотя бы в одной допустимой игровой траектории с учётом объявленного горизонта по количеству ходов. Важно не смешивать это свойство со следующими игровыми характеристиками: средняя продолжительность, справедливость позиций, интересность и доля побед слабой стратегии. Эти характеристики требуют самостоятельных критериев оценки.

4 МЕТОД СТАТИЧЕСКОЙ И СИМУЛЯЦИОННОЙ ДИАГНОСТИКИ

4.1 Архитектура и последовательность проверок

Программа организована как последовательность уровней, имеющих разные задачи и границы применимости:

1. Структурная проверка Pydantic устанавливает, что JSON может быть однозначно интерпретирован как GameRules; несоответствие схеме обрабатывается отдельно.
2. Контрактная проверка ищет явно недопустимые эффекты и цели карт; каждое нарушение сопровождается конкретным диагностическим сообщением.
3. Ресурсная проверка вычисляет верхнюю оценку U максимального суммарного урона за R ходов; если U < H, невозможность штатной наступательной победы строго следует из предпосылок модели.
4. Симуляция агентами Greedy и Random фиксирует возможные причины завершения и при нахождении победы сохраняет seed и траекторию как свидетельство её существования.
5. Модуль принятия решения собирает результаты, не превращая отсутствие побед в доказательство дефекта.

Это отвечает на вопрос, где находится «предлагаемая исследованием проверка»: разработанный вклад представлен контрактной диагностикой, оптимистическим ресурсным анализом и правилами интерпретации результатов. Pydantic нужен как техническая предпроверка, а агенты нужны для поиска примеров поведения.

4.2 Доказуемая верхняя граница урона

Обозначим число собственных ходов R, стартовое здоровье соперника H, урон i-го экземпляра атакующей карты d_i, её стоимость c_i, а бинарную переменную использования экземпляра $z_i \in \{0,1\}$. В оптимистическую задачу оптимизации включаются положительно повреждающие enemy-карты со стоимостью не выше mana_cap. Множество включает все экземпляры таких карт из фиксированной колоды игрока.

Поскольку разрешено не более одного действия за ход и карта не возвращается в колоду, число использованных атак ограничено R. При начальной мане 0 и приросте не более одной единицы за ход общий объём потраченной маны не может превысить R. Любая реальная атакующая последовательность удовлетворяет:

$$\sum_i z_i \le R; \qquad \sum_i c_i z_i \le R; \qquad z_i \in \{0,1\}.$$

Для оценки доступного урона решается оптимизационная задача:

$$U = max \sum_i d_i z_i \quad при\ указанных\ ограничениях.$$

При расчёте U разрешается выбирать атакующие карты так, как будто все они сразу доступны игроку. Не моделируются порядок добора, ограниченность размера руки, последовательность накопления маны, действия соперника и затраты ходов на лечение. Все эти послабления могут только увеличить оптимистическое множество допустимых планов по отношению к настоящему игровому процессу. Поэтому реальный максимальный нанесённый урон не превышает U.

Утверждение. Если U < H, то в рамках установленной игры штатная наступательная победа через прямой урон противнику невозможна за R собственных ходов.

Доказательство. Предположим, что выигрышная атакующая последовательность существует. Она разыгрывает не более R отдельных экземпляров карт и потребляет не более R маны; её вектор z допустим в оптимистической задаче. Следовательно, нанесённый ею суммарный урон не превосходит U. Но для снятия стартовых H HP даже при отсутствии лечения противника необходимо нанести как минимум H урона. При U < H получаем противоречие. Таким образом, искомая последовательность невозможна.

Обратное утверждение не выполняется: U ≥ H не доказывает существования выигрыша, поскольку временной порядок получения карт или накопления маны может помешать реализации оптимистического плана. Поэтому результат «U достаточно для победы» не является автоматическим принятием игровой конфигурации.

4.3 Алгоритм динамического программирования

Вычисление U реализовано динамическим программированием как вариант задачи о «0/1 рюкзаке» с двумя ресурсами. Состояние DP[a,m] задаёт максимальный урон при выборе ровно **a** экземпляров карт общей стоимостью **m**, либо специальное значение для недостижимой комбинации. Начальное значение DP[0,0] = 0. Для каждой копии карты выполняется обновление при обратном обходе счётчиков, исключающее повторное использование её экземпляра:

$$DP[a+1, m+c_i] \leftarrow max(DP[a+1, m+c_i], DP[a,m] + d_i).$$

Рассматриваются только состояния a ≤ R, m ≤ R. Итоговое U — максимум среди допустимых ячеек. Если K — число атакующих экземпляров, время алгоритма составляет

$O(KR^2)$, память — $O(R^2)$. Метод масштабируется лучше полного перебора игровых партий при небольшом количестве ходов и при этом выдаёт формально проверяемую верхнюю границу.

Для каждого отрицательного решения сохраняются исходные значения H и R, число подходящих экземпляров карт, рассчитанное U и текст объяснения. Таким образом, система предоставляет проверяемую причину отказа, а не непрозрачный балл качества.

4.4 Три статуса и смысл результатов симуляции

Таблица 5 – Семантика статусов и границы их интерпретации

Статус	Достаточное основание	Чего статус НЕ означает
PROVEN_INVALID	Явное нарушение игрового контракта либо U < H.	Не утверждается, что такая механика запрещена в любой другой игре.
WITNESSED_OFFENSIVE_WIN	Контракт соблюдён; в симуляторе зафиксирована выигрышная траектория.	Не доказаны баланс, интересность и победа при всех раздачах.
INCONCLUSIVE	Ни нарушения, ни победной траектории найдено не было.	Нельзя заключить, что победы не существует.

Результаты формальных проверок имеют приоритет над симуляцией. Так, в конфигурации heals_enemy игрок может выиграть атакующими картами, но направление лечебного эффекта на соперника нарушает контракт. Найденная победа не устраняет этого факта. Напротив, если нарушений не было найдено и Greedy агент не победил, система не отклоняет конфигурацию, а возвращает INCONCLUSIVE.

Для симуляции используются две фиксированные политики. Random Agent выбирает одну из доступных карт случайно; Greedy Agent предпочитает наиболее сильную из доступных атакующих карт, а при отсутствии атакующих карт вообще может лечиться или добирать карты. Отсутствие выигрыша у одной или обеих политик не исключает третьей, более подходящей стратегии.

5 ПРОВЕДЕНИЕ ЭКСПЕРИМЕНТА

5.1 Реализация и контроль корректности

Прототип системы реализован на Python: Pydantic задаёт структуру GameRules, типы данных описаны с помощью dataclasses, случайность определяется экземпляром random.Random(seed), ресурсная оценка рассчитывается динамическим программированием. Генерация LLM, обращение к интернету и использование GPU отсутствуют. Колоды создаются отдельно для двух игроков; начальная рука извлекается из колоды без дублирования экземпляров.

Для контроля корректности программной модели проведены проверки:

- отклонение неизвестного типа эффекта по Pydantic;
- сохранение количества экземпляров карт при перемещении между колодой и рукой;
- повторяемость симуляций при одинаковом seed;
- правильная раздельная регистрация завершения партий при opponent_hp_zero, self_defeat и turn_limit;
- заранее известных значений U = 4, 6 и 10 для трёх специальных сценариев;
- сопоставление динамического программирования с полным перебором подмножеств атакующих карт на контрольных примерах.

Эти тесты снижают риск ошибок программной реализации, но не являются независимым математическим доказательством корректности всего игрового движка. Вывод о формальном ограничении справедлив лишь при выполнении предпосылок модели, а свидетельство победы — при корректном воспроизведении победной траектории.

5.2 Контрольный набор и методика сравнения

Эксперимент содержит 15 заранее сформированных конфигураций (Таблица 6): 7 допустимых по опубликованному контракту и экспериментальному замыслу и 8 со специально внесёнными нарушениями. Все они проходят структурную проверку. Это осознанный выбор дизайна: в рамках работы сравниваются методы семантической диагностики, а не качество парсинга JSON. Эталонные метки сценариев назначены на основании их описания, не результатов проверки системой.

Таблица 6 – Входные конфигурации и заранее определённые метки

Сценарий	Эталон	Назначение
base	допустима	Базовые правила

Сценарий	Эталон	Назначение
cheaper_attacks	допустима	Бесплатные атаки
stronger_attacks	допустима	Усилены атаки
longer_game	допустима	Увеличен горизонт
no_healing	допустима	Лечение заменено атакой
weak_but_reachable	допустима	Слабее атаки, но победа возможна
strategy_trap	допустима	Greedy не копит ману, победная траектория есть
no_attacks	с дефектом	Нет damage-карт
zero_damage	с дефектом	Атаки с нулевой силой
unaffordable_attacks	с дефектом	Атаки дороже mana_cap
short_horizon	с дефектом	Недостаточно ходов
weak_and_short	с дефектом	Слабые атаки и короткий горизонт
attacks_self	с дефектом	Атаки направлены на self
heals_enemy	с дефектом	Лечение направлено на enemy
mana_bottleneck	с дефектом	Недостаточная сумма получаемой маны

Для каждого сценария выполнено по 200 игр с Greedy против Greedy и 200 игр с Random против Random агентами при фиксированных seed 0–199, всего 15 × 200 × 2 = 6000 основных партий. В отдельном опыте для сценария strategy_trap использован агент wait_for_blast, который пытается накопить ману на сильную карту; эти 200 запусков не включены в основную величину 6000.

Количественное сравнение проводится с тремя реализованными статическими подходами на одних и тех же конфигурациях: (1) только Pydantic; (2) ранние предметные эвристики; (3) предметный контракт вместе с новой ресурсной оценкой. Результаты игровых симуляция сравниваются не по «числу доказанных ошибок», поскольку они не доказывают их отсутствие или существование, а по обнаруженным победным свидетельствам и причинам завершения. Для корректных случаев отдельно проверяется отсутствие ложного доказанного отказа.

6 РЕЗУЛЬТАТЫ И ИХ ИНТЕРПРЕТАЦИЯ

6.1 Общая сравнительная оценка

Все 15 документов прошли Pydantic. Среди восьми дефектных конфигураций одна лишь схема не распознала ни одного семантического нарушения. Ранние статические эвристики обнаружили семь дефектов, а контрактный анализ с новой ресурсной границей — все восемь. При этом ни одна из семи предусмотренных допустимых конфигураций не получила ошибочный формальный отказ.

Таблица 7 – Сравнение методов на одном контрольном наборе

Метод	Обнаружено из 8	Ложные формальные отказы из 7	Интерпретация
Pydantic	0	0	Проверяет структуру
Ранние статические проверки	7	0	Контракт + грубая оценка
Контракт + граница U	8	0	Добавлена ресурсная диагностика

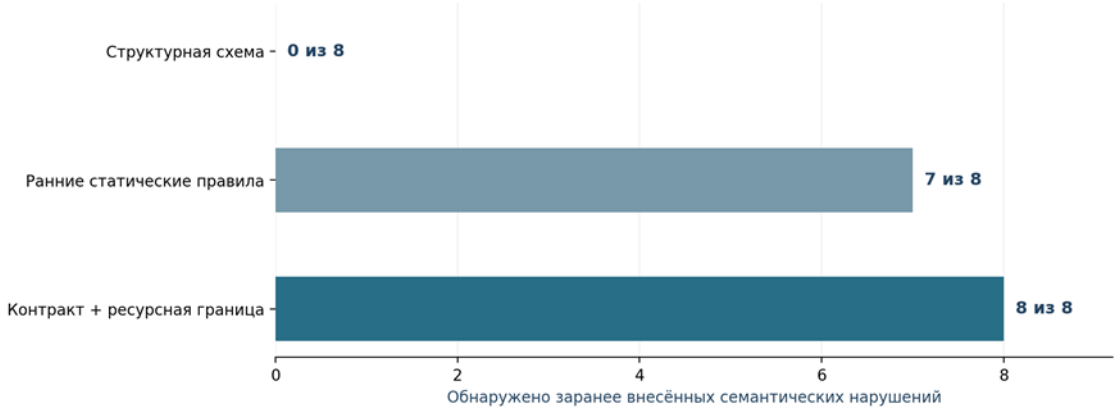


Рисунок 3 – Дополнительная ошибка, найденная ресурсным анализом

Итог трёхзначной процедуры на 15 конфигурациях: PROVEN_INVALID — 8, WITNESSED_OFFENSIVE_WIN — 6, INCONCLUSIVE — 1. Эти числа описывают созданный контрольный набор тестов, а не частоты ошибок в произвольных игровых правилах. Наличие статуса INCONCLUSIVE не засчитывается как ложная отбраковка.

Таблица 8 – Результаты симуляций по всем сценариям

Сценарий	U, HP	Greedy: штатные победы	Статус
base	21	100%	Найдена победа
cheaper_attacks	21	100%	Найдена победа
stronger_attacks	30	100%	Найдена победа

Сценарий	U, HP	Greedy: штатные победы	Статус
longer_game	21	100%	Найдена победа
no_healing	23	100%	Найдена победа
weak_but_reachable	12	100%	Найдена победа
no_attacks	0	0%	Доказан дефект
zero_damage	0	0%	Доказан дефект
unaffordable_attacks	0	0%	Доказан дефект
short_horizon	6	0%	Доказан дефект
weak_and_short	3	0%	Доказан дефект
attacks_self	0	0%	Доказан дефект
heals_enemy	21	100%	Доказан дефект
mana_bottleneck	4	0%	Доказан дефект
strategy_trap	10	0%	Неопределённо

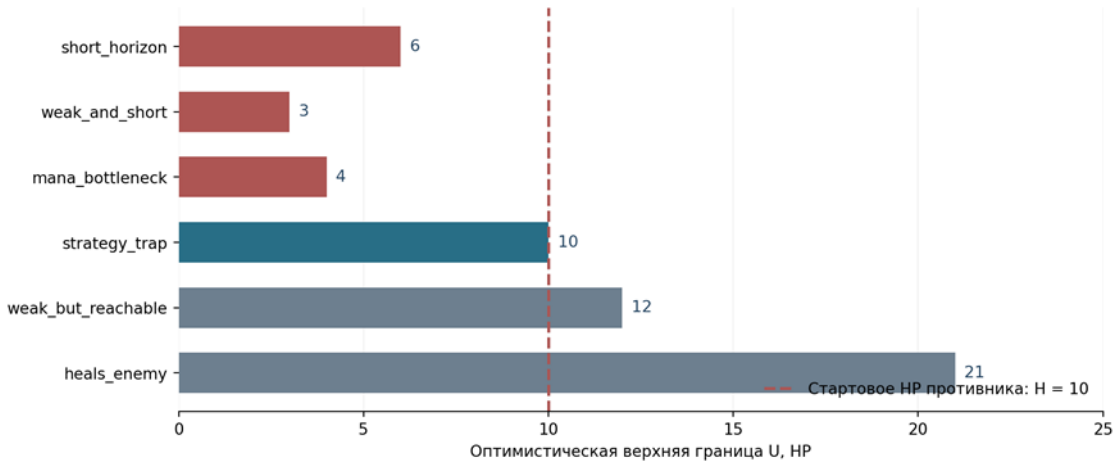


Рисунок 4 – Верхние границы максимального урона для показательных конфигураций

6.2 Доказательство на примере mana_bottleneck

Сценарий mana_bottleneck специально проверяет недостаточность простой эвристики «максимальный урон одной атаки × число ходов», которая использовалась в более ранней версии статической проверки. При H = 10, R = 5 и атакующих картах стоимостью c = 3 и силой d ≤ 4 эта эвристика даёт 5 × 4 = 20 максимального урона и не выявляет проблемы. При этом каждое допустимое использование атаки требует три единицы маны, а за пять собственных ходов её будет начислено всего не более пяти единиц.

Пусть x — целое число оплаченных атак. Тогда 3x ≤ 5, следовательно, x ≤ 1. Урон одной атаки не выше 4 HP, поэтому U = 4 < H = 10. Значит, в пределах пяти ходов штатная наступательная победа невозможна независимо от порядка добора и стратегии.

Динамическое программирование выводит тот же показатель U = 4. Этот результат является строгим в рамках объявленной модели и не зависит от нулевой доли побед Greedy.

Именно здесь получено локальное улучшение относительно ранней проверки: выявлено ресурсное ограничение, которое не выражается проверкой отдельных полей карты и грубой оценкой без суммарной стоимости. Улучшение относится к реализованному базовому подходу, а не к результатам опубликованных систем в работах [5] и [11].

6.3 Контрпример strategy_trap и роль агентов

Сценарий strategy_trap решает другой, логически независимый вопрос. В нём стартовое HP равно 10, горизонт ходов составляет R = 3, а начальная рука включает слабую атаку strike (урон 1, цена 1) и сильную blast (урон 10, цена 3). Первая стратегия Greedy расходует полученную в первом раунде ману на strike. Накопить три единицы к третьему раунду она уже не успевает. В итоге, в 200 партиях Greedy против Greedy штатные победы не наблюдались.

Однако в рамках данной игры разрешён пропуск действия. Альтернативная стратегия wait_for_blast пропускает два собственных хода, получает три единицы маны и в третьем разыгрывает карту blast, сводя HP противника с 10 до 0. Этот результат воспроизводится при seed = 0; в отдельном эксперименте по той же стратегии найдено 200 побед из 200. Поэтому основной статус после симуляций с Greedy — INCONCLUSIVE, а после предъявления победной траектории от альтернативной стратегии можно сделать положительный вывод о существовании выигрыша.

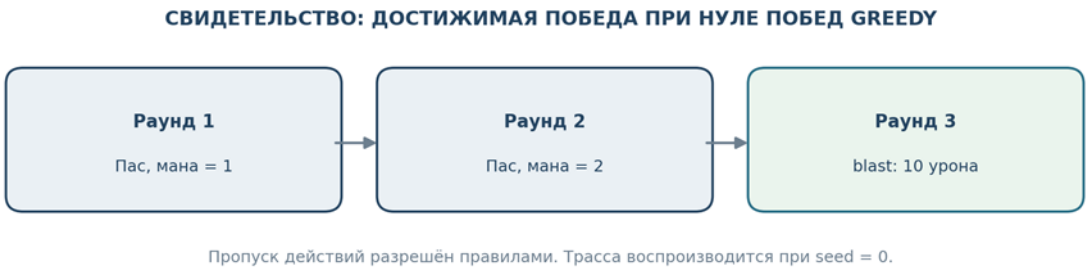


Рисунок 5 – Конструктивная выигрышная траектория для сценария strategy_trap

Этот пример опровергает распространённую ошибку интерпретации автоматических тестов: из 0 побед Greedy нельзя заключить, что выигрышной стратегии нет вообще. Для утверждения «существует выигрыш» достаточно одной проверенной траектории, однако для утверждения «выигрыша нет» нужно исключить все возможные

допустимые траектории или предъявить достаточное формальное ограничение, что и делается в другом сценарии.

6.4 Что действительно доказал эксперимент

В эксперименте подтверждены три положения:

1. На рассматриваемых входах соблюдение JSON-схемы не исключает ни контрактных, ни ресурсных семантических ошибок: все 15 документов структурно валидны, при этом 8 из них заранее имеют дефект.
2. Ресурсная верхняя оценка обобщает частный расчёт для конкретных карт и позволяет доказать недостижимость наступательной победы в mana_bottleneck, где прежняя грубая оценка не даёт отказа.
3. Агентные проверки полезны для поиска успешных игровых путей и исследования игрового поведения, но отрицательное наблюдение требует статуса неопределённости: strategy_trap содержит достижимый выигрыш при нуле побед Greedy.

Дополнительные наблюдения подкрепляют выбранное трёхзначное разделение. В сценарии attacks_self агент Greedy приводит к условию self_defeat, поэтому сам факт завершения партии не является достаточным показателем корректной победы. В сценарии heals_enemy Greedy находит штатную победу, однако правило лечения противника нарушает опубликованный контракт. Для weak_but_reachable Greedy добивается штатной победы в 100% запусков, тогда как Random — в 17%: результат заметно зависит от выбранной политики.

Ограничения статистической интерпретации принципиальны: 200 запусков одной политики по заранее установленным seed не образуют доказательства её наилучшего поведения и не дают универсальную оценку истинной вероятности победы для произвольной популяции игроков. Конструктивное свидетельство с конкретной выигрышной траекторией и формальный сертификат $U < H$ обладают другой, явно указанной логической силой.

7 ОГРАНИЧЕНИЯ И ПРАКТИЧЕСКОЕ ЗНАЧЕНИЕ

Формальная граница U применима лишь при следующих предпосылках: начальная мана равна нулю; прирост не превышает одной единицы за собственный ход; нет альтернативных источников прямого урона и маны; за ход можно сыграть не более одной карты; экземпляр атакующей карты используется не больше одного раза; условие победы — снятие HP противника атакующим действием. Введение возврата карт, ускоренного накопления маны, нескольких действий за ход или комбинаций эффектов требует пересмотра доказательства.

Контрактные нарушения относительны к конкретной спецификации. Так самоповреждающая атака или лечение врага могли бы быть полезны в иной версии игры с другими механиками. Поэтому доказанный контрактный дефект означает «нарушает требование этой модели», а не «это плохая механика в любой игре». Кроме того, нахождение одной победной траектории не даёт оснований говорить о сбалансированности, интересности и отсутствии иных ошибок.

Эксперимент ограничен 15 вручную созданными вариантами одной минимальной карточной дуэли, без независимой экспертной разметки и без воспроизведения алгоритмов сравниваемых научных работ. Валидатор и симулятор разработаны в рамках одного проекта; тестирование отдельных инвариантов снижает, но не устраняет вероятность ошибок общей реализации. Поэтому доли 8/8 и 6/7 нельзя использовать как оценку качества метода на генеральной совокупности игр.

Практическая значимость. Предложенная архитектура показывает, как валидация может быть встроена после любого источника правил — ручного дизайна, шаблонного генератора или впоследствии LLM — без смешения качества генерации и качества проверки. Сначала выполняются недорогие определённые проверки; затем применяется формальный ресурсный запрет; наконец, агентные прогоны помогают найти конкретные свидетельства и обнаружить чувствительность игры к выбранной стратегии. Метод снижает риск необоснованного отбора или отбраковки наборов игровых правил на основании только одной эмпирической метрики.

Перспективы развития. Среди них можно выделить реализацию независимого набора тестов и разметку экспертами, расширение ресурсной модели, поиск по пространству состояний или модельную проверку для более полного анализа достижимости победы, сравнение с различными агентами и отдельное исследование

генераторов правил, если будет возобновлена работа с LLM. Для текущей учебной практики эти задачи не требуются: одна узкая проверяемая проблема уже закрыта.

ЗАКЛЮЧЕНИЕ

В ходе учебной практики на основе ранее разработанного симулятора поставлена и решена узкая задача автоматической диагностики семантических ошибок заранее заданных правил карточной дуэли. Явно разделены структурная корректность документа, соблюдение предметного контракта, динамическая достижимость штатной победы и наблюдаемая успешность тестового агента. Указано, почему их смешение приводит к методологически неверным решениям.

Разработан вычислимый достаточный критерий недостижимости победы через верхнюю границу суммарного урона U. Граница учитывает ограниченное число ходов, суммарную получаемую ману и количество экземпляров карт. Она рассчитана алгоритмом динамического программирования. В конфигурации mana_bottleneck строго доказано $U = 4 < H = 10$ при формально допустимом JSON — нарушение, не распознанное ранней оценкой без учёта общего ресурсного бюджета.

Для устранения ошибок интерпретации агентной оценки введены три статуса: PROVEN_INVALID, WITNESSED_OFFENSIVE_WIN и INCONCLUSIVE. Эксперимент на 15 фиксированных конфигурациях и 6000 основных симуляциях дал восемь доказанных нарушений, шесть случаев найденной Greedy победы и один неопределённый случай. В последнем альтернативная стратегия предъявила победную траекторию, несмотря на отсутствие побед у агента Greedy.

Научно-практический смысл результата заключается не в утверждении о новом универсальном валидаторе, а в демонстрации корректной архитектуры вывода: формальное доказательство недостижимости нельзя подменять неуспехом агента в рамках симуляций, а положительная траектория не доказывает общего качества правил. Работа показывает, какое ограничение реально закрывает реализованный метод и какие вопросы остаются вне его возможностей.

СПИСОК ИСПОЛЬЗОВАННЫХ ИСТОЧНИКОВ

[1] Browne C., Maire F. Evolutionary Game Design // IEEE Transactions on Computational Intelligence and AI in Games. 2010. Vol. 2, No. 1. P. 1–16. DOI: 10.1109/TCIAIG.2010.2041928.

[2] Smith A. M., Nelson M. J., Mateas M. Ludocore: A Logical Game Engine for Modeling Videogames // Proceedings of IEEE Conference on Computational Intelligence and Games. 2010. P. 91–98. DOI: 10.1109/ITW.2010.5593368.

[3] Smith A. M., Mateas M. Answer Set Programming for Procedural Content Generation: A Design Space Approach // IEEE Transactions on Computational Intelligence and AI in Games. 2011. Vol. 3, No. 3. P. 187–200. DOI: 10.1109/TCIAIG.2011.2158545.

[4] Ebner M., Levine J., Lucas S. M., Schaul T., Thompson T., Togelius J. Towards a Video Game Description Language // Artificial and Computational Intelligence in Games. Dagstuhl Follow-Ups. 2013. Vol. 6. P. 85–100. DOI: 10.4230/DFU.Vol6.12191.85.

[5] Zook A., Riedl M. O. Automatic Game Design via Mechanic Generation // Proceedings of the AAAI Conference on Artificial Intelligence. 2014. Vol. 28, No. 1. DOI: 10.1609/aaai.v28i1.8788.

[6] Zook A., Fruchter E., Riedl M. O. Automatic Playtesting for Game Parameter Tuning via Active Learning // Proceedings of the 9th International Conference on the Foundations of Digital Games. 2014. URL: https://faculty.cc.gatech.edu/~riedl/pubs/zook-fdg14.pdf (дата обращения: 25.10.2026).

[7] Shaker N., Togelius J., Nelson M. J. Procedural Content Generation in Games. Cham: Springer, 2016. 237 p. DOI: 10.1007/978-3-319-42716-4.

[8] Osborn J., Lambrigger B., Mateas M. HyPED: Modeling and Analyzing Action Games as Hybrid Systems // Proceedings of the AAAI Conference on Artificial Intelligence and Interactive Digital Entertainment. 2017. Vol. 13, No. 1. P. 87–93. DOI: 10.1609/aiide.v13i1.12937.

[9] Yannakakis G. N., Togelius J. Artificial Intelligence and Games. Cham: Springer, 2018. 337 p. DOI: 10.1007/978-3-319-63519-4.

[10] Holmgård C., Green M. C., Liapis A., Togelius J. Automated Playtesting with Procedural Personas through MCTS with Evolved Heuristics // IEEE Transactions on Games. 2019. Vol. 11, No. 4. P. 352–362. DOI: 10.1109/TG.2018.2808198.

[11] Gonzalez J. J., Cooper S., Guzdial M. Mechanic Maker 2.0: Reinforcement Learning for Evaluating Generated Rules // Proceedings of the AAAI Conference on Artificial Intelligence and Interactive Digital Entertainment. 2023. Vol. 19, No. 1. P. 266–275. DOI: 10.1609/aiide.v19i1.27522.

[12] Aylagas M. V., Bergdahl J., Gillberg J., Sestini A., Tolstoy T., Gisslén L. Improving Conditional Level Generation Using Automated Validation in Match-3 Games // IEEE Transactions on Games. 2024. Vol. 16, No. 4. P. 783–792. DOI: 10.1109/TG.2024.3440214.

ПРИЛОЖЕНИЕ А. ПРИМЕР ВХОДНОГО ДОКУМЕНТА

В качестве минимального машиночитаемого контракта используется схема GameRules, реализованная средствами Pydantic. Ниже дан полный пример одного входа с ресурсной ошибкой:

```
{
  "game_name": "Fixed Card Duel",
  "max_rounds": 5,
  "starting_hp": 10,
  "mana_cap": 3,
  "starting_hand_size": 3,
  "deck_size": 12,
  "win_condition": "hp_zero",
  "cards": [
    {"name": "strike", "cost": 3, "effect": "damage", "value": 3, "target": "enemy"},
    {"name": "blast", "cost": 3, "effect": "damage", "value": 4, "target": "enemy"},
    {"name": "restore", "cost": 2, "effect": "heal", "value": 2, "target": "self"},
    {"name": "insight", "cost": 1, "effect": "draw", "value": 1, "target": "self"}
  ]
}
```

Схема задаёт формат данных, но не устанавливает возможности накопить достаточный для победы ресурс. Результат ресурсной проверки данного документа: U = 4, H = 10, статус PROVEN_INVALID, причина «невозможность наступательной победы по верхней границе».

ПРИЛОЖЕНИЕ Б. ВОСПРОИЗВОДИМОСТЬ

Исполняемый эксперимент содержит файл validation_proof_aware.py и эквивалентный Jupyter Notebook validation_proof_aware.ipynb. Данные сохранены в validation_results_proof_aware.csv, примеры диагностических сертификатов — в proof_certificates.json, выигрышная траектория — в strategy_trap_witness.json. Для запуска достаточно Python 3.10+ и pydantic>=2. Локальная программа не использует LLM, интернет или графический ускоритель.

Команда запуска: python validation_proof_aware.py. При повторном запуске с одинаковыми seed и версией программы ожидается совпадение результатов. Реализованные сценарии формируются функцией scenarios(), а численные итоги могут быть сверены с сохранённым CSV.

Таблица 1 - Выигрышная траектория strategy_trap для seed = 0

Раунд	Игрок	Действие	Мана перед действием	HP соперника после
1	0	пас	1	10
1	1	пас	1	10
2	0	пас	2	10
2	1	пас	2	10
3	0	blast	3	0